

Game-Theoretic Approach to LTLf Planning and Synthesis

Giuseppe De Giacomo

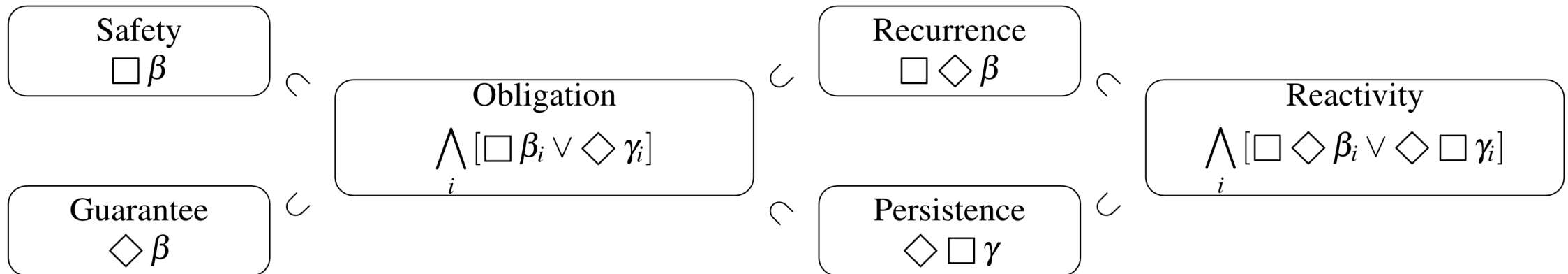
University of Oxford

REVISITING INFINITE TRACES

Interpreting LTLf and PPLTL on Infinite Traces

- Viewed from the perspective of infinite trace, LTLf and PPLTL, easily specify two classes of properties:
 - **guarantee properties**
 - something good eventually happens (typically used for expressing temporally extended goals that need to be fulfilled in finite time)
 - by requiring that there **exists** a finite prefix of infinite traces that satisfies the LTLf/PPLTL formula.
 - **safety properties**
 - nothing bad ever happens (typically used for expressing declarative control knowledge in AI),
 - by requiring that **all** finite prefixes of infinite traces satisfy the LTLf/PPLTL formula.
- These two classes are at the bottom level of a classic hierarchy of LTL properties called the **safety-progress** hierarchy.

Safety-Progress Hierarchy



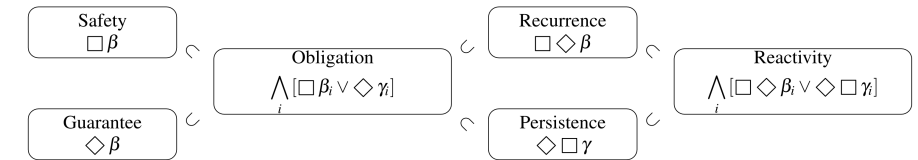
where β, γ are PPLTL formulas.

Safety-Progress Hierarchy

The safety-progress hierarchy was introduced by Lichtenstein, Pnueli and Zuck in 1985 later described in detail by Manna and Pnueli in 1989.

It consists of six classes of semantic properties, organized by inclusion.

- The first level contains
 - **safety properties**
 - **guarantee properties** (sometimes also called co-safety properties);
- The second level consists of the class of **obligation properties**, i.e., positive Boolean combination of safety and guarantee properties;
- The third level contains,
 - **recurrence properties** (that say that something good occurs infinitely often),
 - **persistence properties** (that say that nothing bad occurs infinitely often);
- The fourth level contains the **reactivity properties**, i.e., positive Boolean combinations of recurrence and persistence properties.



Each property is semantically defined in terms of sets of finite-traces, originally expressed in PPLTL.

Safety-Progress Hierarchy

- An **infinite-trace property** P is (i.e., denotes) a set of infinite traces.
- A **finite-trace property** F is a set of finite traces.

Classification

- An infinite-trace property P is **safety property** if there is a finite state property F such that for all infinite traces $\pi \in P$ **all** non-empty finite prefixes of π are in F .
- An infinite-trace property P is **guarantee property** if there is a finite state property F such that for all infinite traces $\pi \in P$ there **exists** a non-empty finite prefix of π in F .
- An infinite-trace property P is **obligation property** if it can be expressed as a **boolean combination** of **safety** and **guarantee** properties.
- An infinite-trace property P is **recurrence property** if there is a finite state property F such that for all infinite traces $\pi \in P$ we have that **infinitely many prefixes** of π are in F .
- An infinite-trace property P is **persistence property** if there is a finite state property F such that for all infinite traces $\pi \in P$ **all but finitely many prefixes** of π are in F .
- An infinite-trace property P is **reactivity property** if it can be expressed as a **boolean combination** of **recurrence** and **persistence** properties.

- An **LTL <class> property** is a <class> property that can be expressed in LTL.
- Thm: Every **first-order <class> property** is an LTL <class> property, and viceversa.

Safety-Progress Hierarchy

Manna and Pnueli consider the LTL-definable properties in each class of the hierarchy.

When restricted to LTL-definable properties, the class at the top of the hierarchy, the reactivity properties, contain all properties expressible in LTL.

They provide syntactic characterizations of each such class using pure past LTL formulas (PPLTL)

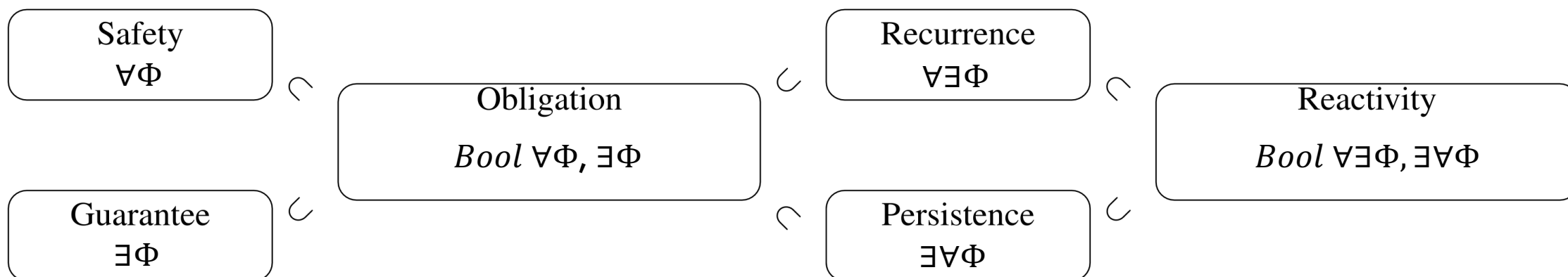
In this characterization the logic PPLTL plays a key role, though later in 1992, Chang, Manna, and Pnueli presented an alternative elegant characterization of the safety-progress hierarchy in terms of standard LTL..

Let ϕ be an arbitrary PPLTL formula:

- safety properties are expressible with formulas of the form $\Box\phi$,
- guarantee properties as formulas of the form $\Diamond\phi$,
- obligation properties as positive Boolean combinations of $\Box\phi$ and $\Diamond\phi$,
- recurrence properties as formulas of the form $\Box\Diamond\phi$,
- persistence properties as formulas of the form $\Diamond\Box\phi$, and
- reactive formulas as Boolean combination of $\Box\Diamond\phi$ and $\Diamond\Box\phi$.

The temporal operators $\Box$ and $\Diamond$ act as quantifiers over the finite prefixes of an infinite trace ($\Box$ says “for every prefix”, $\Diamond$ says “for some prefix”); on such prefixes we check the given property of finite-traces.

Safety-Progress Hierarchy



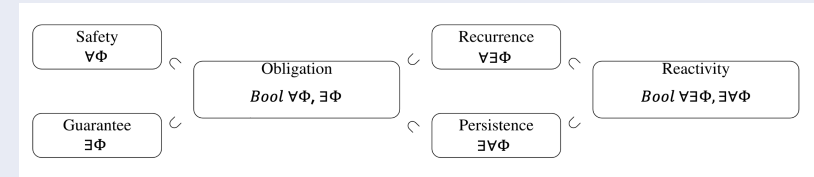
LTL_f+ & PPLTL+ on Infinite Traces

LTL_f+ and PPLTL+- Syntax

The syntax of LTL_f+ / PPLTL+ is given by the following grammar, where Φ are formulas in LTL_f / PPLTL:

$$\Psi ::= \forall\Phi \mid \exists\Phi \mid \forall\exists\Phi \mid \exists\forall\Phi \mid \Psi \vee \Psi \mid \Psi \wedge \Psi \mid \neg\Psi$$

- Formulas Φ are called **finite-trace formulas**.
- Formulas $\forall\Phi, \exists\Phi, \forall\exists\Phi, \exists\forall\Phi$ are called **infinite-trace formulas**.



Note: being $\forall\Phi \mid \exists\Phi$ and $\forall\exists\Phi \mid \exists\forall\Phi$ we may put LTL_f+ / PPLTL+ in positive normal form (in linear time) and drop $\neg\Psi$.

LTL_f+ and PPLTL+- Semantics

- Infinite trace π satisfies $\forall\Phi$ if **all prefixes** $\pi^k = \pi_1, \dots, \pi_k \models \Phi$ – express **safety**.
- Infinite trace π satisfies $\exists\Phi$ if **some prefix** if some (non-empty) finite prefix $\pi^k = \pi_1, \dots, \pi_k \models \Phi$ – express **guarantee**.
- Infinite trace π satisfies $\forall\exists\Phi$ if **infinitely many prefix** $\pi^k = \pi_1, \dots, \pi_k \models \Phi$ – express **recurrence**.
- Infinite trace π satisfies $\exists\forall\Phi$ if **finitely many prefix** $\pi^k = \pi_1, \dots, \pi_k \not\models \neg\Phi$ – express **persistence**.
- Positive boolean combination of $\forall\Phi$ and $\exists\Phi$ express **obligation**.
- Positive boolean combination of $\forall\exists\Phi$ and $\exists\forall\Phi$ express **reactivity**.

LTLf+ & PPLTL+ on Infinite Traces

- Planning domains, non-Markovian domains, and safety properties more generally, can all be expressed with formulas of the form $\forall \Phi$. For example — below ϕ_x are propositional formulas:

- Conditional effect axioms can be expressed in LTL_f+ and PPLTL+ by

$$\forall \Box (\phi_c \supset (\bigcirc a \supset \bigcirc \phi_e)) \qquad \forall \Box (\Theta \phi_c \supset (a \supset \phi_e))$$

- Frame axioms can be expressed as

$$\forall \Box (\phi_c \wedge \bigcirc a \supset (p \equiv \bigcirc p)) \qquad \forall \Box (\Theta \phi_c \wedge a \supset (\Theta p \equiv p))$$

- To predicate about the initial state we can include propositional formulas ϕ_{init} as abbreviations:

$$\forall \phi_{init} \qquad \forall (\Box (first \supset \phi_{init}))$$

- Fairness of effects, e.g., assuming that action a under condition ϕ_c can effect two effects ϕ_{e_1} or ϕ_{e_2} , is expressed as

$$\forall \Box \Diamond (\phi_c \wedge \bigcirc (a \wedge last)) \supset \bigwedge_i \forall \Box \Diamond (\phi_c \wedge \bigcirc (a \wedge last \wedge \phi_{e_i})) \qquad \forall \Box ((\Theta \phi_c) \wedge a) \supset \bigwedge_i \forall \Box ((\Theta \phi_c) \wedge a \wedge \phi_{e_i})$$

- Arbitrary LTL_f temporally extended goals Φ can be expressed using formulas of the form $\exists \Phi$, e.g., eventually reaching a state where ϕ_g holds while guaranteeing to maintain a safety condition ϕ_s can be expressed as

$$\exists (\Diamond \phi_g \wedge \Box \phi_s) \qquad \exists (\phi_g \wedge \Box \phi_s)$$

Theorem (expressive power)

The logics LTL_f+, PPLTL+, and LTL define the same infinite-trace properties.

LTLf+ & PPLTL+ on Infinite Traces

We begin with a formula Ψ in positive normal-form. The formula Ψ can be thought of as a positive Boolean formula $\widehat{\Psi}$ over the infinite-trace formulas $Q_i(\Phi_i)$, for $i \in [k]$ for some k , where $Q_i \in \{\exists, \forall, \exists\forall, \forall\exists\}$.

Step 1: obtain the DFA of the finite trace formulas

Convert each finite-trace formula Φ_i into an equivalent DFA $A_i = (D_i, F_i)$. Say $D_i = (\Sigma, Q_i, \iota_i, \delta_i)$.

Step 2: create the components of the game

For each $i \in [k]$, build a transition system $D'_i = (\Sigma, Q_i, \iota_i, \delta'_i)$, a set $F'_i \subseteq Q_i$, and an objective O_i over Q_i :

- Say $Q_i = \forall\exists$. Let $\delta'_i = \delta_i$, $F'_i = F_i$, and require that a run of D'_i satisfies O_i iff it has infinitely many prefixes ending in a final state.
- Say $Q_i = \exists\forall$. Let $\delta'_i = \delta_i$, $F'_i = F_i$, and require that a run of D'_i satisfies O_i iff it does not have infinitely many prefixes ending in a non-final state, i.e., almost all its prefixes end in a final state.
- Say $Q_i = \forall$. If ι_i is not a final state, make it into a final state, i.e., define $F'_i = F_i \cup \{\iota_i\}$. Let δ'_i be like δ_i except that every non-final state is a sink; and require that a run of D'_i satisfies O_i iff all its prefixes end in a final state.
- Say $Q_i = \exists$. If ι_i is a final state, make it into a non-final state, i.e., define $F'_i = F_i \setminus \{\iota_i\}$. Let δ'_i be like δ_i except that every final state q is a sink; and require that a run of D'_i satisfies O_i iff at least one of its prefixes ends in a final state.

Call a state $q_i \in Q_i$ "marked" if either $Q_i \neq \exists\forall$ and $q_i \in F'_i$, or $Q_i = \exists\forall$ and $q_i \in Q_i \setminus F'_i$.

Step 3: combine components into a game arena with Emerson-Lei objectives

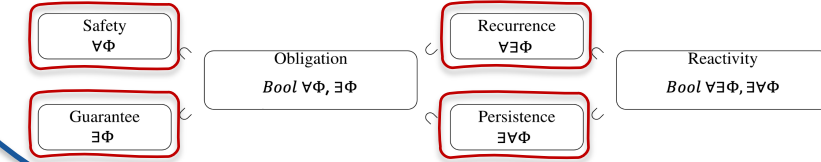
Build the product transition system $D = \prod_{i \in [k]} D'_i$. Let Q be the state set of D . Define an Emerson-Lei condition (Γ, λ, B) over Q as follows: $\Gamma = [k]$ (i.e., the labels are the component numbers), $\lambda(q) = \{i \in [k] : q_i \text{ is marked}\}$ (i.e., a state is labeled by the numbers of those components that are in marked states), and the Boolean formula B is formed from the Boolean formula $\widehat{\Psi}$ by simultaneously replacing, for every i with $Q_i = \exists\forall$, every occurrence of i by $\neg i$. Define the deterministic Emerson-Lei automaton $A_\Psi = (D, (\Gamma, \lambda, B))$.

Step 4: solve the game

Solve the Emerson-Lei game A_Ψ .

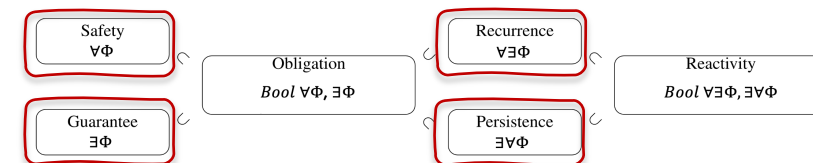
Emerson-Lei games can be solved in time polynomial in the size of the arena and exponential in the number of labels.

The size of A_Ψ is that of its components, i.e., 2exp for LTLf+ and 1exp for PPLTL+, hence we are optimal.



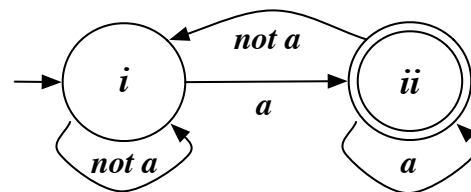
Reduction to game:
Game arena built with the same techniques as for LTLf/PPLTL
Game objectives on infinite traces

Example



This is the DFA for the formula Φ on finite traces saying “at last a ”

Φ (on finite traces)

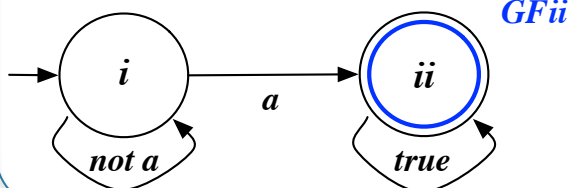


These are the Emerson-Lei deterministic automata for $\exists\Phi$, $\forall\Phi$, $\forall\exists\Phi$, $\exists\forall\Phi$

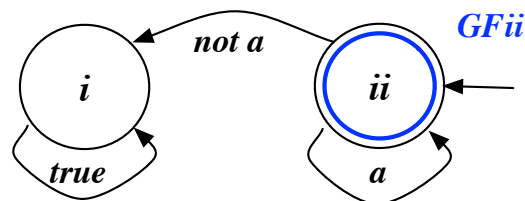
Taking $\wedge$ and $\vee$ of the Emerson-Lei deterministic automata returns Emerson-Lei deterministic automata

Emerson-Lei deterministic automata correspond to Emerson-Lei games that can be solved in poly in the number of states and exp in number of labels

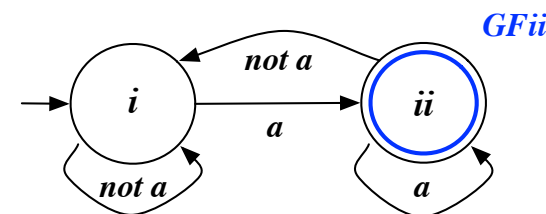
$\exists\Phi$



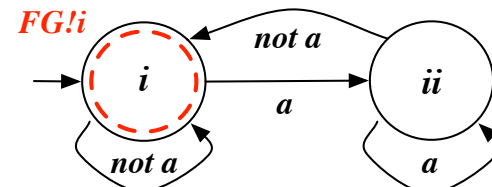
$\forall\Phi$



$\forall\exists\Phi$



$\exists\forall\Phi$



LTLf+ & PPLTL+ on Infinite Traces

Theorem (expressive power)

The logics LTL_{f+} , PPLTL+ , and LTL define the same infinite-trace properties.

Theorem (synthesis)

- LTL_{f+} synthesis is 2EXPTIME -complete.
- PPLTL+ synthesis is EXPTIME -complete.

Key point: technique based on reduction to games:

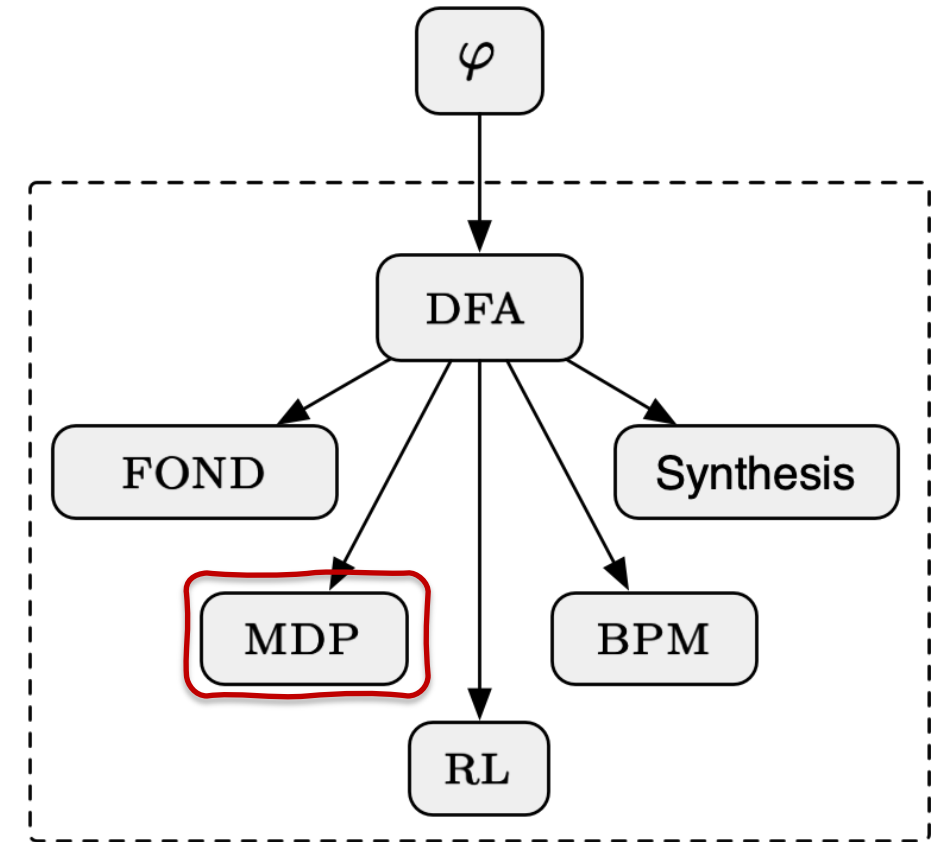
- **Game arena** built with **DFA**s as for LTLf/PPLTL
- **Game objectives on infinite traces**

MDPS

MDPs with LTLf+ and PPLTL+ Goals

Many Applications:

- Reactive Synthesis
- Planning for temporally extended goals
- **MDP with non-Markovian goals**
- Reinforcement Learning for non-Markovian tasks and safety constraints
- Declarative Process Specification in BPM
- ...



MDPs

Stochastic model of the environment: Markov Decision Process (MDP)

$$M = (S, A, T, L)$$

where:

- S is the state space
- A is the action space
- $T : S \times A \rightarrow \text{Prob}(S)$ is the transition function
- $L : S \rightarrow 2^{\text{Prop}}$ is the labeling function

Note no goal/reward, these are given separately as a linear temporal formula φ .

Objective: find a strategy $\sigma: S^* \rightarrow A$ that maximizes the probability of satisfying φ .

Solving MDPs for LTL Goals

classical solution

Solving MDP for linear temporal logic goal φ

1. Build nondeterministic Büchi automaton (NBA) for φ (exp)
2. Convert the NBA to a DRA (exp)
(This is the notorious determinization of omega-regular automata!)
3. Construct product MDP $M \times A$ (poly)
4. Identify accepting end components (ECs) of $M \times A$ (poly)
5. Compute (memoryless) strategy with max probability of reaching accepting ECs (poly)

Solving MDPs for LTL Goals

current state-of-the-art solution

Solving MDP for linear temporal logic goal φ

1. Build nondeterministic Büchi automaton (NBA) for φ (exp)
2. Convert the NBA to a “good for MDPs (GFM)” Limit Deterministic BA (LDBA) (exp)
(LDBAs are nondeterministic, but being GFM such nondeterminism can be resolved by the agent, without loss of generality!)
3. Construct product MDP $M \times A$ (poly)
4. Identify accepting end components (ECs) of $M \times A$ (poly)
5. Compute (memoryless) strategy with max probability of reaching accepting ECs (poly)

Gives the burden of resolving (angelic) nondeterminism to the agent!

Notice that the agent does not have clairvoyance, so it chooses blindly!

When an automaton is GFM then this blind choice is good enough to get the optimal solution!

Solving MDPs for LTLf+/PPLTL+ Goals

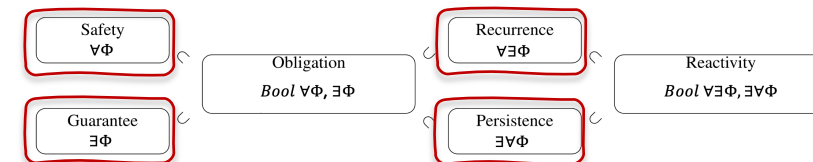
New solution!

Solving MDP for LTLf+/PPLTL+ goal φ

1. Build the DFAs of the finite components of φ (2exp)
2. Convert each DFAs into “good for MDPs” LDBA (poly)
3. Obtain A by taking $\wedge$ and $\vee$ (according to φ) of the LDBAs (poly in states, exp in number of $\wedge$ and $\vee$) - *Thm: A is good for MDPs!*
4. Construct product MDP $M \times A$ (poly)
5. Identify accepting end components (ECs) of $M \times A$ (poly)
6. Compute (memoryless) strategy with max probability of reaching accepting ECs (poly)

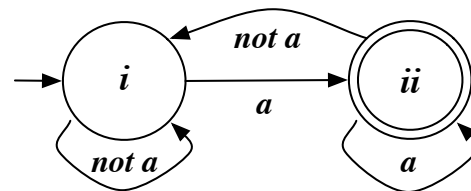
** Paper under submission*

Example



This is the DFA for the formula Φ on finite traces saying “at last a ”

Φ (on finite traces)

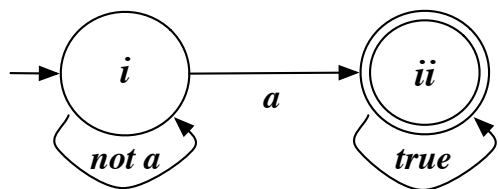


Taking $\wedge$ and $\vee$ of these GFM returns GFM automaton A

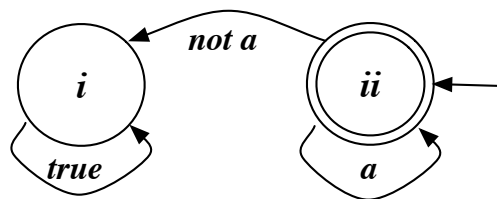
$M \times A$ can be solved in poly in the number of states by standard techniques!

These are the GFM, LDBA for $\exists\Phi, \forall\Phi, \forall\exists\Phi, \exists\forall\Phi$

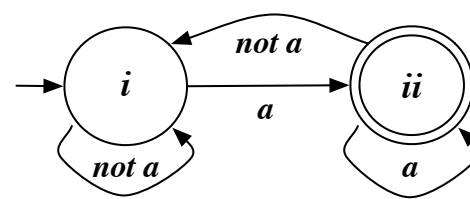
$\exists\Phi$



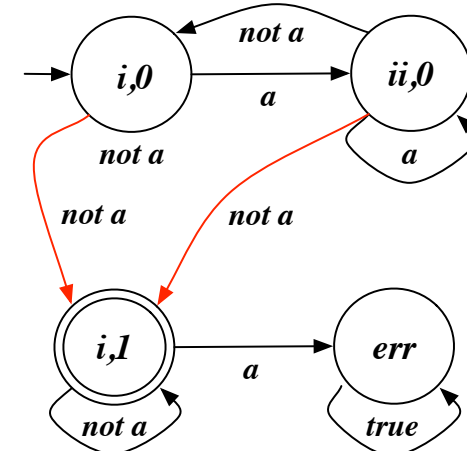
$\forall\Phi$



$\forall\exists\Phi$



$\Phi \wedge E$



CONCLUSIONS

Conclusions

- **LTLf and PPLTL are a success story** for synthesis, planning, MDPs and reinforcement learning
 - **LTLf reactive synthesis** scales orders of magnitude better than LTL synthesis
 - **PPLTL goals** and safety constraints can be handled by standard planners with **minimal overhead** [ICAPS2023, ECAI 2023, ECAI2024]
 - **PPLTL shielding** in Reinforcement Learning also has minimal overhead [AAAI2024]
- We now have a path to **extend this success story to infinite traces!**
 - **LDLf+/PPLDL+**
 - All results extend to omega regular languages or **MSO** –vs. **FOL** - on infinite traces of LTL
- Are we done? Not yet!
 - **For synthesis**, we still have to **solve infinite games**, which require parity conditions or nested fixpoints – very new solutions coming from Formal Methods – *joint work with Liverpool!*
 - For MDPs things are easier, but implementation is needed – *again, joint work with Liverpool!*
 - We have **removed** what has been considered **the major bottleneck**: the construction of the **game arena (product MDP)**

